

geo3d no cartucho V9968 padrão: relatório de verificação

Data: 27/09/2026. Base: projeto do cartucho do HRA! (Tang Nano 20K, FPGA Gowin GW2AR-18C), versão 86361d8, com o geo3d integrado pelo script `geo3d/integration/apply_geo3d_patch.py`. Ramo `geo3d-phase2`. Tudo desta etapa está no GitHub (alexmoncks/V9968_Cartridge): ramo `geo3d-phase2` em `eb9be4f` e `main` em `a72a761`. A organização das pastas da seção 4 (o projeto do HRA! intocado em `fpga/V9968_Cartridge_TangNano20K/` e o projeto com o geo3d ao lado, em `fpga/V9968_Cartridge_TangNano20K_geo3d/`) veio depois desses commits.

1. Resumo

Pergunta	Resposta
Precisa mudar a placa física?	Não. Nenhum componente, trilha, pino, cristal ou jumper.
Muda o projeto original do HRA!?	Não, a pasta dele fica exatamente como no upstream. O geo3d fica num segundo projeto do Gowin ao lado, gerado por script a partir do dele, onde 5 arquivos dele recebem o patch (só FPGA).
Cabe no FPGA do cartucho?	Sim. Gowin oficial: 62% da lógica, 86% dos slices, 48% da memória, 38% dos DSPs, 6 de 8 clocks primários.
O geo3d tem clock próprio?	Sim. O <code>clk42g</code> (42,95 MHz) sai do mesmo PLL do <code>clk85m</code> . O <code>clk42m</code> do HRA! e o HDMI ficaram como estavam.
A lógica funciona?	Sim, em simulação. A placa inteira do HRA! foi simulada com o geo3d e o clock novo. A VRAM saiu idêntica à referência em todos os casos.
O tempo (timing) fecha?	Sim, no modelo do Gowin. Zero caminhos com folga negativa em setup, hold, recovery e removal, em todos os pares de clocks, com as configurações do próprio projeto do HRA!. Algumas folgas são justas (seção 6).
A ROM de demos detecta o geo3d?	Sim. Sem ele, mostra "geo3d não encontrado" em três idiomas e fica parada ali, em vez da tela preta.
Já pode gravar no cartucho?	Sim. O bitstream <code>geo3d_cartridge_86361d8.fs</code> (cópia de <code>fpga/V9968_Cartridge_TangNano20K_geo3d/impl/pnr/tangnano20k_vdp_cartridge.fs</code>) e as instruções <code>FLASH.txt</code> (inglês, português e japonês) estão prontos. Grave com o cartucho fora do MSX.
Testado no cartucho real?	Não. Nada aqui substitui o teste no hardware (seções 10 e 11).

2. O que foi verificado e como

Item	Método	Resultado
Placa e slot	Esquemas e netlist em <code>pcb/</code> , RTL <code>msx_slot.v</code> , simulação de ciclos de E/S do Z80	Funciona sem modificação
Script de integração	Gerador rodado sobre o 86361d8 limpo do HRA!, comparado com o projeto do geo3d no repositório; segunda execução; <code>--check</code> ; os 46 testes de <code>geo3d/integration/test_apply_geo3d_patch.sh</code> (texto do HRA! mudado, inclusive uma nova fonte de leitura no mux do barramento e sinais renomeados ou com outra largura; pasta do HRA! já modificada; edições à mão recusadas e preservadas, <code>--force</code> com cópia de segurança; atualizações do projeto do HRA! e de <code>geo3d/rtl</code> ; arquivos sobrando e faltando; configurações mudadas; um clone só do repositório do HRA!)	Árvore idêntica (fora o fim de linha CRLF do Windows); a segunda execução não grava nada; os arquivos modificados são idênticos byte a byte aos do patch anterior, aplicado na própria pasta; 46 de 46 testes passam
Ocupação e tempo	Gowin EDA Standard V1.9.12.03 (licença do Alex), build completo do projeto do geo3d na própria pasta, como o Run All da IDE, com as configurações do projeto (Place Option 0). Tabelas de caminhos de setup e hold para cada par de clocks, tiradas de uma cópia que gera o mesmo bitstream	Cabe; zero violações
Ferramentas abertas	Yosys + nextpnr-himbaechel (oss-cad-suite)	O original roteia; com o geo3d não posiciona (98,6%). Não servem de referência para este chip
Lógica integrada	iverilog: top completo do HRA!, VDP, slot, controlador de SDRAM e modelo Micron, mais o geo3d, com o <code>clk42g</code> , tudo tirado do <code>src/</code> do projeto do geo3d. Também com o <code>clk42g</code> atrasado 0,5 ns e 2,0 ns. A bancada de teste fica fora do repositório (no ambiente de trabalho do autor)	Passou em tudo
Equivalência do VDP	Prova formal (Yosys) e simulação lado a lado	VDP modificado idêntico ao original com o geo3d parado
ROM de demos	Simulador Z80 <code>geo3d/rom/run_rom_z80.py</code> , com e sem geo3d, 88h e 98h	256 de 256 execuções passam
Software sem geo3d	openMSX sem geo3d: 88h e 98h, MSX1, MSX2 PAL, MSX2+, sem cartucho	ROM BASIC e jogo corretos; ROM de demos mostra "geo3d não encontrado"

3. Placa (hardware)

Conclusão: o cartucho padrão atende o geo3d sem nenhuma modificação. A correção de clock (seção 4) é só no FPGA.

- **Endereços:** A0-A7 chegam ao FPGA pelo U5 (SN74LVC8T245, DIR em GND, só entrada). A8-A15, /SLTSL, /MERQ e /M1 não chegam: o cartucho é só de E/S, e as ROMs ficam em outro slot.
- **Controle:** /IORQ, /RD, /WR e /RESET pelo U6 (DIR em GND).
- **Dados:** o U4 tem a direção comandada pelo FPGA (`SLOT_DATA_DIR`, com pull-down de 10k em R2).
- **/BUSDIR:** sai pelo Q1 (NMOS em dreno aberto), com o gate no mesmo `SLOT_DATA_DIR`. Fica baixo enquanto o cartucho dirige o barramento, para qualquer porta.
- **/WAIT:** pelo Q2, só durante a partida (até o FPGA configurar e a SDRAM inicializar). O geo3d não usa /WAIT.
- **Decodificação:** o `msx_slot.v` original já decodifica o bloco inteiro de 8 portas (88h-8Fh ou 98h-9Fh, pela chave DIP). O VDP usa +0 a +4, e o geo3d usa +5 (índice e status) e +7 (dados). O +6 (8Eh) fica de fora de propósito.
- **Leitura:** as leituras do geo3d saem pelo mesmo caminho das leituras de status do VDP, com a mesma virada do U4 e o mesmo /BUSDIR.
- **Tempo de resposta simulado:** o geo3d põe o dado no barramento 116 a 140 ns depois do /RD (139,7 ns na simulação com o clk42g). O Z80 a 3,58 MHz lê cerca de 500 ns depois, então sobra folga larga, e ainda sobram cerca de 220 ns a 7,16 MHz. O atraso dos conversores de nível não entra nessa conta.

Pontos a conferir no seu equipamento: - **MegaRAM:** segundo o mapa de portas do msx.org (não reconferido), 8Eh-8Fh são usadas pela MegaRAM. Uma MegaRAM que decodifique só parte do endereço pode reagir às leituras do geo3d em 8Fh. - **Cristal:** a lista de peças da placa (`pcb/.../parts.txt`) indica 28,636 MHz para o U1, mas os PLLs do projeto esperam 14,318 MHz. Vale olhar qual está montado. Isso vale igualmente para o bitstream original do HRA!. - **Alimentação:** o pino de 5 V do Tang Nano está ligado ao +5 V do slot. **Grave o bitstream com o cartucho fora do MSX.**

4. O que muda no projeto do HRA!

A pasta do HRA!, `fpga/V9968_Cartridge_TangNano20K/`, não muda: é exatamente o upstream 86361d8 dele, com o build dele em `impl/` (o bitstream de recuperação), então os merges das atualizações dele nunca dão conflito. O geo3d fica num segundo projeto do Gowin ao lado, `fpga/V9968_Cartridge_TangNano20K_geo3d/`, organizado como o dele e com os mesmos nomes de arquivo:

Caminho em <code>fpga/V9968_Cartridge_TangNano20K_geo3d/</code>	Conteúdo
<code>tangnano20k_vdp_cartridge.gprj</code>	A lista de arquivos dele mais <code>src/geo3d/geo3d_core.v</code> , <code>geo3d_engine.v</code> e <code>geo3d_bus.v</code> (caminhos dentro do projeto, então a IDE do Gowin abre o projeto como está)
<code>src/</code>	Os arquivos que o <code>.gprj</code> dele lista, mais os arquivos do gerador de IP (<code>.ipc</code> , <code>.mod</code> , <code>_tmp.v</code> , <code>.vo</code>) dos IPs dele. Quatro deles recebem o patch abaixo
<code>src/geo3d/</code>	Cópias de <code>geo3d/rt1</code> , que continua sendo o único lugar para editá-los
<code>impl/tangnano20k_vdp_cartridge_process_config.json</code>	As configurações do projeto dele, sem mudança (Place Option 0)
<code>impl/gwsynthesis/</code> , <code>impl/pnr/</code>	O nosso build do Gowin, no git do mesmo jeito que o dele. O bitstream é <code>impl/pnr/tangnano20k_vdp_cartridge.fs</code>
<code>geo3d_manifest.txt</code> , <code>README.md</code>	Gravados pelo script: os arquivos que ele gerou, com o SHA-256 do texto de cada um (para distinguir o que ele gerou de edições à mão), e uma descrição curta da pasta

O `apply_geo3d_patch.py` gera essa pasta a partir da pasta do HRA! e de `geo3d/rt1`, e só lê a pasta dele. Ele é idempotente (uma segunda execução não grava nada) e não grava nada se o texto do HRA! mudar ou se a pasta dele já tiver mudanças do geo3d. A cada execução, ele confere a configuração do rPLL2, a ligação do `u_p112` (entrada clk14m, saída clk85m), os clocks do SDC, o mux de leitura do barramento (tem de ser exatamente o do upstream, para que uma nova fonte de leitura nunca seja descartada), a declaração e a largura de cada sinal dele em que o geo3d se liga, e as portas do slot e do VDP ligadas a esses sinais. Ele não sobrescreve edições à mão no projeto do geo3d (feitas na IDE do Gowin, por exemplo, que mostra as cópias em `src/geo3d`): recusa e diz onde a edição deve ir, e o `--force` guarda uma cópia dos arquivos ao lado da pasta antes de sobrescrevê-los. O `--check` não grava nada e confirma que a pasta é exatamente o que o script gera, em especial que `src/geo3d` é igual a `geo3d/rt1`. Os arquivos modificados são idênticos byte a byte aos do patch anterior, aplicado na própria pasta (conferido nesta data). O script precisa de Python 3 (Linux, macOS, WSL ou Windows); o `geo3d/integration/test_apply_geo3d_patch.sh` testa o script em cópias temporárias.

Arquivo do HRA! (modificado no projeto do geo3d)	Mudança (linhas, contra o 86361d8)	Efeito
<code>src/v9968/vdp.v</code>	+11 / -3: uma porta externa de escrita nos registradores de comando (<code>ext_cmd_wr/num/data</code>) somada à da CPU, e a saída do CE	Com o geo3d parado, o VDP é idêntico ao original: prova formal (695 de 695 pontos; 697 de 697 no 86361d8) e simulação lado a lado de 3,6 milhões de ciclos com todos os pinos iguais
<code>src/tangnano20k_vdp_cartridge.v</code> (top)	+44 / -4: instancia o <code>geo3d_bus</code> nas portas +5 e +7, tira essas portas do VDP, junta os dados de leitura, cria o fio <code>clk42g</code> , liga <code>u_p112</code> a ele e leva o <code>clk_eng</code> do geo3d ao <code>clk42g</code>	O VDP continua respondendo em +0 a +4, e o +6 continua indo ao VDP
<code>tangnano20k_vdp_cartridge.gprj</code>	+3: os arquivos do geo3d (<code>src/geo3d/</code>)	Nenhum

Arquivo do HRA! (modificado no projeto do geo3d)	Mudança (linhas, contra o 86361d8)	Efeito
src/gowin_rpll2/gowin_rpll2.v	+3 / -3: expõe a saída <code>clkoutd</code> . Ela já estava configurada no PLL (CLKOUT / 2, DYN_SDIV_SEL = 2) e ficava sem uso	Nenhum para o HRA!: CLKOUT e CLKOUTP não mudam
src/tangnano20k_vdp_cartridge.sdc	+7: <code>create_generated_clock</code> do <code>clk42g</code> (<code>clk14m</code> x 3, como o <code>clk85m</code> é <code>clk14m</code> x 6) e <code>set_clock_uncertainty 0.5</code> nas duas direções entre <code>clk85m</code> e <code>clk42g</code> , com comentários	Só a análise de tempo

Não mudam no projeto do geo3d: `msx_slot.v`, o arquivo de pinos (`.cst`), o HDMI, a SDRAM, o `clk42m` do HRA! (continua saindo do divisor CLKDIV e alimentando o HDMI e o resto do projeto dele), os clocks e grupos de clocks do SDC do HRA!, e as configurações do projeto. A pasta do próprio HRA! não muda em nada; o `impl/pnr/` dela guarda o bitstream original dele para recuperação.

Cuidado: os arquivos `.ipc` e `.mod` do IP do rPLL2 são copiados sem mudança. Se alguém regenerar esse IP no Gowin, no projeto do geo3d, a porta `clkoutd` some e a síntese falha com erro. Nesse caso, rode o script de novo.

IDE do Gowin: na primeira vez que a IDE abre o projeto do geo3d, ela cria `impl/temp/rtl_parser.result` e `impl/temp/rtl_parser_arg.json` e regrava o `tangnano20k_vdp_cartridge.gprj.user`, como na pasta do HRA! (que guarda esses arquivos no git). O `gw_sh`, usado pelo `build_gowin.sh`, não os cria, então eles ainda não estão no repositório.

Arquivos mudados nesta etapa:

Arquivo	Mudança
fpga/V9968_Cartridge_TangNano20K/	De volta ao upstream 86361d8 do HRA!, arquivo por arquivo
fpga/V9968_Cartridge_TangNano20K_geo3d/ (novo)	O projeto do geo3d gerado pelo script, com o build do Gowin em <code>impl/</code>
geo3d/integration/apply_geo3d_patch.py	Gera o projeto do geo3d a partir da pasta do HRA!, em vez de modificar a pasta dele, com <code>--check</code> ; os dois passos novos (rPLL2 e SDC), o <code>clk42g</code> no top, as conferências descritas acima e a proteção das edições à mão (<code>geo3d_manifest.txt</code> , <code>--force</code>)
geo3d/integration/test_apply_geo3d_patch.sh (novo)	Os 46 testes do script, em cópias temporárias (bash e Python 3: Linux ou WSL)
geo3d/syn/gowin/build_gowin.sh (novo)	Faz o build do projeto do geo3d na própria pasta, como o Run All da IDE, com as configurações do projeto. Roda no Git Bash; antes, roda o <code>apply_geo3d_patch.py --check</code> (com o python3 do WSL se o Windows não tiver Python) e para se o projeto não estiver em dia. As tabelas de caminhos saem de um segundo build numa cópia fora do repositório, cujo SDC recebe os comandos de relatório, para todos os pares dos clocks que o SDC do projeto define; essa cópia tem de gerar o mesmo bitstream. Termina com código 2 se houver qualquer folga negativa
geo3d/syn/cartridge/synth_cartridge.sh	A síntese no Yosys gera o projeto do geo3d na pasta de trabalho dela, a partir do projeto do HRA! (basta um clone do repositório dele, como no <code>run_all.sh</code>) e do <code>geo3d/rtl</code> do próprio script
geo3d/rtl/geo3d_bus.v	Só o comentário do cabeçalho (<code>clk42g</code> e a margem do SDC). Nenhuma lógica mudou
geo3d/rom/geo3d_rom.asm, build_rom.py, run_rom_z80.py	Detecção do geo3d na ROM de demos e novos modos do simulador (seção 8)
geo3d/docs/BASIC_API.md	Regra R#32-R#58 (seção 9)

5. Ocupação do FPGA (Gowin, GW2AR-LV18QN88C8/I7)

Builds sobre a base 86361d8, Gowin V1.9.12.03, configurações do projeto do HRA!. Os relatórios do Gowin do build final estão em `fpga/V9968_Cartridge_TangNano20K_geo3d/impl/`; os resumos e as tabelas de caminhos, em `openmsx-geo3d/fpga/reports\` (pasta de entrega, fora do repositório).

Recurso	Original HRA!	Com geo3d, clk42m (antes)	Com geo3d, clk42g (final)	Diferença final
Lógica (LUT + ALU)	7.023 (34%)	12.756 (62%)	12.756 (62%)	+5.733
Registradores	4.219 (27%)	7.536 (48%)	7.536 (48%)	+3.317
Slices (CLS)	5.605 (55%)	8.896 (86%)	8.886 (86%)	+3.281
Memória em bloco (BSRAM)	10 de 46 (22%)	22 (48%)	22 (48%)	+12
DSP	3 de 24 (13%)	9 (38%)	9 (38%)	+6
Pinos de E/S	39 de 66	39 de 66	39 de 66	0
Clocks primários	4 de 8	5 de 8	6 de 8	+2
Long wires	5 de 8	8 de 8	8 de 8	+3
PLLs (rPLL) / divisores (CLKDIV)	2 de 2 / 1 de 8	2 de 2 / 1 de 8	2 de 2 / 1 de 8	0

- O clock próprio do geo3d não gasta lógica nem PLL: ele usa uma saída que o PLL já tinha. Gasta uma rede de clock primária (o clk42g roda nos quadrantes TR, TL e BL).
- Os long wires estão em 8 de 8: não sobra nenhum para mudanças futuras.

6. Tempo (timing)

Resultado: zero caminhos com folga negativa em todas as tabelas, com as configurações do próprio projeto do HRA! (Place Option 0). Isso cobre setup, hold, recovery e removal em todos os pares de clocks. O `build_gowin.sh` faz o build do projeto do geo3d na própria pasta exatamente como o Gowin IDE faz (`open_project` e `run_all`, sem opções nem tabelas extras). As tabelas de caminhos saem de um segundo build numa cópia cujo SDC recebe os comandos de relatório; o script confere que ela gera o mesmo bitstream (só a linha "Created Time" muda). O bitstream também é o mesmo que foi entregue antes da mudança na organização das pastas.

Folga pior em ns (positivo = passa). Nas duas colunas com geo3d, o clk42m ou o clk42g é o clock do motor do geo3d.

Verificação (meta)	Original HRA!	geo3d com clk42m (antes)	geo3d com clk42g (final)
Fmax clk85m (85,909 MHz)	86,73 MHz	86,14 MHz	86,34 MHz
Fmax clk42m (42,955 MHz)	120,92 MHz	62,64 MHz	117,46 MHz
Fmax clk42g (42,955 MHz)	não há	não há	64,07 MHz
Setup 85 para 85	+0,111	+0,031	+0,058
Setup 85 para 42m (HDMI)	+1,334	-1,130 (50 ou mais falhando)	+0,489
Setup 85 para 42g	não há	não há	+2,383
Setup 42m para 85	não há	+12,698	não há
Setup 42g para 85	não há	não há	+7,857
Setup 42m para 42m	+15,010	+7,316	+14,767
Setup 42g para 42g	não há	não há	+7,673
Hold 85 para 85	+0,199	+0,227	+0,216
Hold 85 para 42m (HDMI)	+3,443	+3,458	+3,782
Hold 85 para 42g	não há	não há	+0,047
Hold 42m para 85	não há	-2,487 (40 falhando)	não há
Hold 42g para 85	não há	não há	+0,039
Hold 42m para 42m	+0,539	+0,074	+0,544
Hold 42g para 42g	não há	não há	+0,074
Recovery / removal	+10,046 / +1,047	+9,476 / +1,200	+9,476 / +1,185
Caminhos negativos, todas as tabelas	0	90 ou mais	0

- As tabelas listam no máximo 50 caminhos. Por isso "50 ou mais".
- Na coluna do clk42m, as linhas "85 para 42m" também cobrem o motor do geo3d, e os piores caminhos são dele, não do HDMI: o setup vai do reset `ff_reset3_n2` do HRA! ao motor, e o hold 42m para 85 vai dos registradores `cmd_*_e` do `geo3d_bus` aos `cmd_*`.
- As 74 tabelas são setup e hold para os 36 pares dos seis clocks que o SDC do projeto define (clk85m, clk42m, clk42g, clk215m, clk e clk14m; o `build_gowin.sh` os lê do SDC), mais recovery e removal. No build final, 60 delas não têm caminhos: todas as de pares com clk215m, clk ou clk14m (54), as entre clk42m e clk42g (4, porque o HDMI e o geo3d não se ligam) e as de clk42m para clk85m (2).
- As folgas entre clk85m e clk42g já descontam a margem de 0,5 ns. Sem ela, os holds dessa passagem ficariam em cerca de +0,54 ns.
- **Cuidado ao ler o relatório do Gowin:** o resumo "Total Negative Slack" mostra 0 mesmo com caminhos falhando entre clocks. Só as tabelas de caminhos mostram as violações. O `build_gowin.sh` gera essas tabelas e confere todas.

Causa das violações antigas: o clk42m sai de um divisor CLKDIV, que no modelo do Gowin não tem atraso, enquanto o clk85m, que sai do PLL, tem de 2,9 a 4,4 ns. A passagem entre os domínios do geo3d supõe clocks alinhados.

Correção aplicada: o geo3d ganhou o clk42g, da saída CLKOUTD do mesmo PLL do clk85m (CLKOUT / 2). Com isso as passagens do geo3d aparecem com 0 ns de defasagem de clock no modelo (antes, 4,36 ns).

Margem de 0,5 ns: o Gowin dá ao CLKOUTD exatamente o atraso do CLKOUT e não modela o atraso do próprio divisor. O SDC reserva 0,5 ns (`set_clock_uncertainty`) nas duas direções entre clk85m e clk42g. **Esse valor é suposto, não medido.** Para testar a lógica, a simulação também rodou com o clk42g atrasado 0,5 ns e 2,0 ns (seção 7).

Builds intermediários (resumos em `reports\compare\`):

Build	Posicionamento	Margem	Caminhos negativos
clk42m (antigo)	Option 0	não	90 ou mais
clk42g	Option 0	não	1 (caminho do HDMI do HRA!, 85 para 42m, -0,059 ns)

Build	Posicionamento	Margem	Caminhos negativos
clk42g	Option 1	não	0
clk42g (entregue)	Option 0 (padrão do HRA!)	0,5 ns	0
clk42g	Option 1	0,5 ns	0

Folgas justas, mas positivas: - Setup 85 para 85: +0,058 ns, num caminho do motor de comandos do VDP (no original, +0,111 ns; no build com clk42m, +0,031 ns). A folga muda com o posicionamento. - Hold dentro do motor do geo3d (42g para 42g): +0,074 ns. - Holds da passagem 85 e 42g: +0,039 e +0,047 ns, já com a margem. - O caminho do HDMI (85 para 42m) varia com o posicionamento: de -0,059 a +0,962 ns nos builds com clk42g acima. Uma mudança futura na RTL pode movê-lo de novo. O `build_gowin.sh` acusa isso (código 2).

7. Simulação da RTL integrada

Simulado em iverilog o top completo do HRA! (o Verilog do `src/` do projeto do geo3d: 86361d8 limpo mais o script) com o VDP, o slot, o controlador de SDRAM e o modelo Micron MT48LC2M32B2, mais o geo3d, a 85,909 MHz. Os stubs cobrem só os PLLs, o divisor (que divide de verdade por 2) e o HDMI. O stub do PLL gera o `clkoutd` como $\text{clk85m} / 2$, em fase. Um verificador (`clkcheck.v`) confirma que o `clk_eng` do geo3d é o clk42g, na metade exata da frequência, com 0 bordas desalinhadas. O lado do Z80 usa ciclos reais de E/S (/IORQ, /RD, /WR, A0-A7, D0-D7) na velocidade do OTIR. A VRAM (256 KB) começa preenchida com um padrão pseudoaleatório, para pegar qualquer escrita perdida.

Teste	Resultado
Registradores, status e transformações imediatas	0 erros
Arame (cubo e octaedro, com XOR e com TIMP)	VRAM idêntica
Faces sólidas sombreadas	VRAM idêntica
Faces texturizadas (LRMM)	VRAM idêntica
Demo real de texturas (GEO3DT.COM), todas as OUTs reproduzidas	2.842 comandos (1.855 LINE, 987 LRMM), 0 diferentes, 3 quadros idênticos
Chave DIP em 98h	Passou
Estado de partida (modo V9958)	VRAM idêntica
Handshake em 39.158 escritas do geo3d no VDP	0 escritas com CE = 1, 0 perdidas, 0 colisões
Conflito proposital: a CPU escreve R#32-R#58 com o geo3d ocupado (33 escritas)	4 colisões no mesmo ciclo e quadro corrompido, como esperado. Confirma a regra da seção 9

- **Comparação com o clock antigo e com a entrega anterior:** os logs de comandos e de leituras, as VRAMs e a contagem de ciclos saíram idênticos byte a byte aos da simulação com o clk42m e aos da entrega anterior com o clk42g. O teste de estado de partida, que antes só tinha rodado com o clk42m, foi refeito na árvore final nesta revisão.
- **Nova organização das pastas:** os testes de registradores, arame, faces, texturas, handshake, DIP em 98h, registradores em 98h e a demo real rodaram de novo com todos os arquivos tirados de `fpga/V9968_Cartridge_TangNano20K_geo3d/src` (o geo3d de `src/geo3d`). Os 8 passam, com os logs de comandos e de leituras, as VRAMs e a contagem de ciclos idênticos byte a byte aos das execuções anteriores.
- **Clock atrasado:** 7 testes com o clk42g atrasado 0,5 ns e 2,0 ns em relação ao clk85m (14 execuções). Numa simulação sem atrasos de porta, isso faz toda captura de clk85m para clk42g pegar o valor lançado na mesma borda. As 14 passaram. A VRAM saiu idêntica à dos clocks alinhados, menos no teste de conflito proposital, em que as colisões mudam (1 em vez de 4) e, com elas, o quadro corrompido.
- **Bancada de teste:** o `tb_cart.sv`, os stubs, o gerador de ciclos do Z80 e a reprodução da demo ficam fora do repositório (no ambiente de trabalho do autor), então essas execuções não podem ser repetidas só com o repositório.
- **Limite:** a simulação não modela os atrasos reais de roteamento. Eles ficam por conta da análise de tempo (seção 6) e do cartucho real.

8. Nosso software com o bitstream original do HRA! (sem geo3d)

Programa	O que acontece	Situação
ROM BASIC (G3BASIC.ROM)	Abertura "geo3d BASIC 0.2 (none)"; <code>CALL G3INIT</code> dá "Device I/O error"; o BASIC continua funcionando	Correto
Jogo VECTOR RAID	Não trava, mas roda sem a nave e sem os inimigos, sem avisar	Aceitável
ROM de demos	Mostra a tela "geo3d não encontrado" (inglês, espanhol e português) e fica parada nela. Não escreve em nenhuma porta do geo3d	Correto (corrigido)

Como a ROM de demos detecta o geo3d (`geo_probe` em `geo3d_rom.asm`, o mesmo método do `detect` da ROM BASIC, só na base da própria ROM): 1. Na ROM de 98h, num MSX1 (MSXVER = 0), não faz nada: o TMS9918 entenderia a escrita em R#15 como R#7. 2. Lê o ID do VDP em S#1. ID 0, 2 ou 3 quer dizer que há um V99x8, e a imagem da mensagem pode ir para ele. O geo3d exige ID 2 ou 3. 3. Com R#15 = 2, lê P+5. FFh (bitstream do HRA! ou porta vazia) quer dizer sem geo3d. Bits 3-2 = 11 quer dizer um VDP com as portas repetidas em P+4 a P+7. Só depois lê o PORT#4. 4. A primeira escrita no geo3d é o índice 40h. Seguem 17 leituras de P+7, e a 17ª tem de ser igual à primeira.

A detecção faz 29 acessos às portas. Com o geo3d presente, o tráfego depois dela é idêntico ao da ROM anterior.

Sem geo3d: - Se há um V99x8 na base, aparece uma imagem SCREEN 5 no estilo do menu. Ela usa só registradores que qualquer V99x8 aceita: nenhum registrador do V9968, nenhum comando, nenhuma escrita no PORT#4. - A ROM de 88h também escreve o texto na tela do próprio MSX pelo BIOS (INITXT e CHPUT, ASCII simples). É a única mensagem quando não há V9968 em 88h. A de 98h só faz isso num MSX1. - A ROM fica num laço com interrupções desligadas (`ng_stay`). - **50/60 Hz:** a ROM de 98h mantém a frequência da máquina (`R#9 = (RG9SAV & 02h) | 80h` ; 82h numa máquina PAL). A de 88h usa 80h (60 Hz) para o HDMI do cartucho.

Prazo nas esperas: as esperas pelo RUN do geo3d e pelo CE do VDP desistem depois de cerca de 3 s num Z80 a 3,58 MHz (3,08 a 3,29 s no simulador). Num turbo R, a INIT do cartucho roda no Z80; em modo R800 com ROM, cada volta de 65.536 leituras da espera (cerca de 1,6 s num Z80) leva cerca de 0,7 s (medição da revisão, não refeita). Quando o prazo acaba (`hw_timeout`): 1. A música para. 2. O motor de comandos recebe STOP (`R#46 = 0`). 3. A ROM espera o RUN do geo3d terminar, até 32.768 leituras de P+5 (cerca de 0,55 s), para nada do quadro em andamento ser desenhado por cima da mensagem. 4. Um segundo STOP, só se o RUN terminou. 5. A mesma mensagem.

Simulador Z80 (`run_rom_z80.py`), ROMs novas:

Conjunto	Execuções	Resultado
Sem MoonSound (PSG, SCC, OPL, OPL4; 3 idiomas; roteiros de teclas; barra de espaço; 88h e 98h)	120	120 passam. O traço é o dos 29 acessos da detecção seguido do traço de referência, byte a byte
Mensagem (<code>--absent ff/mirror/v9938/none, --stuck geo/ce/cegeo, --msxver, --pal</code> , MSX1, MoonSound, 88h e 98h)	84	84 passam. A CPU termina em <code>ng_stay</code> , a página 0, a paleta e os registradores batem com a imagem, o texto do BIOS confere
MoonSound (640, 256, 128 e 0 KB)	52	52 passam. As escritas por porta são detecção mais referência em 45 de 52; as outras 7 só diferem no fim, onde o tempo real corta a execução

- O modo `--stuck cegno` reproduz o caso do CE preso com o geo3d esperando por ele: 1.001 leituras de status e 2 STOPS, tudo antes da imagem. A ROM anterior falha nesse modo e no `--pal` .

openMSX (sem tela, imagens geradas dos dumps de VRAM, registradores e paleta):

Máquina e cartucho	ROM	Resultado
FS-A1WSX com o V9968 do HRA! sem geo3d	88h	Imagem no V9968 (HDMI) e texto na tela do MSX. Único acesso ao geo3d: uma leitura de 8Dh (FFh)
C-BIOS_V9968_JP sem geo3d	98h	Imagem. Uma leitura de 9Dh (FFh), nenhuma escrita
FS-A1WSX sem cartucho	88h	Só o texto do BIOS; nenhum acesso a 8Dh-8Fh
MSX1 (Expert XP-800) com o V9968 do HRA!	88h	Imagem no V9968 e texto na tela do MSX
FS-A1WSX (V9958 em 98h)	98h	Imagem no V9958
MSX1	98h	Só o texto do BIOS; nenhum acesso a 98h-9Fh
Philips NMS 8245 (PAL, V9938)	98h	Imagem com <code>R#9 = 82h</code> (50 Hz mantido)
Philips NMS 8245 com o V9968 do HRA! sem geo3d	88h	Imagem no V9968 com <code>R#9 = 80h</code>
Com geo3d (88h e 98h)	ambas	Menu e crawl normais; contagem de acessos às portas e VRAM iguais às da entrega anterior

As imagens estão em `openmsx-geo3d\geo3d_not_found\` (9 arquivos).

9. Regras para quem programa o geo3d

- Durante um desenho do geo3d (RUN ocupado), nada pode escrever em **R#32 a R#58**. O LRMM usa `R#47 a R#58`, e uma escrita da CPU no mesmo ciclo de uma escrita do geo3d se perde. O teste de conflito proposital da seção 7 mostra isso.
- **CE = 0 não quer dizer motor livre:** o CE cai entre um comando do geo3d e o seguinte. Espere o bit 0 do status do geo3d.
- A janela do LRMM (`R#51 a R#58`) precisa cobrir a VRAM inteira, que é o valor do reset.

Essas regras estão na seção 7.2 do `BASIC_API.md` . As três ROMs já as respeitam. A única exceção é o STOP do `hw_timeout` na ROM de demos, e só em teoria: o geo3d não escreve registradores com o CE alto nem quando está travado.

10. O que só o cartucho real confirma

- **O atraso real entre CLKOUT e CLKOUTD no chip.** A margem de 0,5 ns é suposta, não medida.
- **As folgas justas da seção 6** (+0,039 a +0,074 ns nos holds, +0,058 ns no setup 85 para 85): a análise de tempo é um modelo.
- **Integridade de sinal:** atrasos dos LVC8T245, ruído e o /BUSDIR um pouco mais tardio em máquinas com buffer de slot, expansores e o R800.
- **SDRAM real** sob o tráfego de comandos do geo3d.
- **HDMI em monitores reais.**
- **Consumo, aquecimento e estabilidade** em uso longo.
- **A tela "geo3d não encontrado" no hardware,** com o bitstream original do HRA!.

11. Roteiro do primeiro teste no hardware

Arquivos na pasta de entrega `openmsx-geo3d` (fora do repositório):

Arquivo	Conferência
fpga\geo3d_cartridge_86361d8.fs	SHA-256 00238660f28d4fa73232fcfd3553b3c1eb554bb1a81b35e52ebfbab3b1e8c3ca, igual ao fpga/V9968_Cartridge_TangNano20K/impl/pnr/tangnano20k_vdp_cartridge.fs do repositório. Mesmo conteúdo da entrega anterior (49a3fc51...): só a linha "Created Time" muda
fpga\recovery\tangnano20k_vdp_cartridge_HRA_86361d8.fs	SHA-256 9957d2b507897370c402973c2a358adad9675c0bb7b45b38f75d1fb6a6b32b73, igual ao fpga/V9968_Cartridge_TangNano20K/impl/pnr/tangnano20k_vdp_cartridge.fs do repositório (pasta do HRA!)
fpga\FLASH.txt	Instruções em inglês, português e japonês
GE03D_88_hardware_real_MOD.ROM	SHA-1 79fbbd4f29f2f3ebffa1e9da414b7917d0ca575f
GE03D_98.ROM	SHA-1 0b8ff70cd6f6ad3dbbcb194ec269987536f42987

Os valores de SHA-256 dos dois bitstreams são os dos arquivos com fim de linha CRLF, como o Gowin grava e como o git os coloca num checkout no Windows. Num checkout em Linux ou macOS o fim de linha é LF e os hashes mudam (geo3d 4ce281b2..., HRA! 7fc1fffa...), com o mesmo bitstream.

As duas ROMs usam a música Star Wars (MOD) e não vão para o git. As outras ROMs da pasta são builds antigos, sem a detecção.

- Referência:** com a chave DIP em 88h, o HDMI ligado e o bitstream **original** do HRA! gravado, rode a ROM BASIC. CALL G3INIT deve dar "Device I/O error". Depois rode GE03D_88_hardware_real_MOD.ROM: deve aparecer "geo3d não encontrado" no HDMI e o texto na tela do MSX. Isso prova que o cartucho, o caminho em 88h e a mensagem funcionam.
- Gravar o bitstream do geo3d, com o cartucho fora do MSX** e ligado ao PC pela USB-C. Confira o SHA-256 antes.
 - Gowin Programmer: dispositivo GW2AR-18C, Access Mode = External Flash Mode, Operation = exFlash Erase, Program thru GAO-Bridge, arquivo geo3d_cartridge_86361d8.fs.
 - Ou: openFPGAloader -b tangnano20k -f geo3d_cartridge_86361d8.fs.
- Primeira partida:** tire o cabo USB, chave DIP em 88h, cartucho no slot com o MSX desligado, HDMI ligado, ligue o MSX. Ele deve iniciar normalmente. Se congelar sem partida, o /WAIT ficou preso: o FPGA não configurou ou a SDRAM não inicializou. Desligue, tire o cartucho e grave de novo.
- ROM BASIC primeiro**, porque é a mais segura: ela só lê até identificar o geo3d, tem esperas com prazo e aceita CTRL+STOP. A abertura deve mostrar "(88h)". Depois rode o exemplo: 10 SCREEN 5:CALL G3INIT, 20 CALL G3OBJ(1,1), 30 CALL G3SPIN(1,1,2,0), 40 CALL G3FRAME:GOTO 40.
- Depois o jogo:** a nave 3D deve aparecer no título.
- Por último a ROM de demos** (GE03D_88_hardware_real_MOD.ROM): menu de idiomas e as demos.
 - "geo3d não encontrado" logo na partida: o cartucho responde, mas sem o geo3d. Confira se este bitstream foi gravado e se a chave DIP está em 88h.
 - A mesma mensagem no meio de uma demo: o geo3d ou o motor de comandos parou de responder por cerca de 3 s. Anote a demo e o momento: é um problema real para investigar.
- Observar:** pontos ou lixo durante o desenho (SDRAM), travamentos depois de alguns minutos e aquecimento. Se possível, testar em outro modelo de MSX, inclusive um PAL e um turbo R.
- Para voltar ao original:** o mesmo procedimento, com o cartucho fora do MSX, usando recovery\tangnano20k_vdp_cartridge_HRA_86361d8.fs (ou o fpga/V9968_Cartridge_TangNano20K/impl/pnr/tangnano20k_vdp_cartridge.fs do HRA!, com o mesmo conteúdo).

12. Pendências

- Primeiro teste no cartucho real** (seção 11). É o único jeito de confirmar a margem de 0,5 ns e as folgas justas.
- geo3d/README.md** (somente leitura de propósito, precisa da aprovação do Alex): os parágrafos de integração e de tempo ainda descrevem a organização antiga (o patch aplicado na pasta do HRA!, o build do Gowin numa cópia).
- geo3d/demos/README.md, README.pt.md e README.es.md:** o parágrafo "Hardware real" e o uso do run_rom_z80.py (opções novas --absent, --stuck, --msxver e --pal) ainda descrevem o estado anterior.
- ROMs antigas na pasta de entrega:** GE03D_88_hardware_real.ROM, as *_crawlonly, GE03D_98_fmconv e GE03D_98_midi não têm a detecção. Refazer ou apagar.
- Nomes dos menus do Gowin Programmer** no FLASH.txt: vêm do procedimento da Sipeed para o Tang Nano 20K e não foram conferidos nesta máquina.
- Espaço em disco:** as pastas de experimentos C:\Projects\mmsoft\g3x (561 MB) e geo3d_review_gw* estão fora do repositório e podem ser apagadas.